

Linux ARM64 内存管理

Linux 6.18.1 / ARM64 / QEMU 1GB
 VA_BITS=52, PA_BITS=52, PGTABLE_LEVELS=5, CONFIG_NUMA=y,
 CONFIG_TRANSPARENT_HUGEPAGE_ALWAYS=y
 Linux ARM64 内存管理
 内存管理 = 物理 + 虚拟 + QEMU 内存管理

内存管理

```
graph TD
    Z["0: head.S→start_kernel"]
    A["1: ARM64 内存管理  
MMU + Cache + TLB"]
    B["2: 内存块"]
    C["3: ARM64 内存管理  
PGD→PUD→PMD→PTE"]
    D["4: 内存块"]
    E["5: struct page/folio"]
    F["6: Buddy 内存管理"]
    G["7: SLUB 内存管理"]
    H["8: VMA 与 mmap"]
    I["9: 内存管理  
Demand Paging 内存管理"]
    J["10: 内存管理  
kswapd + LRU + watermark"]
    K["11: 内存管理  
THP/Swap/OOM/CMA"]
    L["12: 内存管理  
物理 + 虚拟"]

    Z --> A
    A --> B
    B --> C
    C --> D
    D --> E
    E --> F
    F --> G
    G --> H
    H --> I
    I --> J
    J --> K
    K --> L
```

```
style Z fill:#9C27B0,color:#fff
style A fill:#4CAF50,color:#fff
style C fill:#FF9800,color:#fff
style F fill:#2196F3,color:#fff
style G fill:#2196F3,color:#fff
style I fill:#FF9800,color:#fff
style J fill:#f44336,color:#fff
```

```
0head.S → start_kernel 3
```

```
|
|_____
|   |                                     ARM64 0000000000000000
|
|_____|
|   |
|   |
|   |
|   |
|   | 001: MMU OFF → 00 Identity Map
|   |
|   |
|_____
|   |
|   | primary_entry:
|   |   |
|   |   | x0 = FDT 00000000bootloader 0000
|   |
```

```
x19 = MMU 00000000
```

```
x21 = FDT 00
```

```
bl create_init_idmap ← 0 init_idmap_pg_dir 00000000
```

```
VA = PA000000: _stext → _end
```

```
text: PAGE_KERNEL_ROX00000000
```

```
data: PAGE_KERNEL000000
```

```
bl __cpu_setup ← 00 TCR/MAIR/SCTLR00MMU0000
```

```
b __primary_switch ← 00 MMU 0000
```

▼

002: MMU ON → 00000000

```
__primary_switch:
```

```
TTBR0 ← init_idmap_pg_dir 0000000000=000000
```

```
TTBR1 ← reserved_pg_dir 0000000000000000
```

```
set_sctlr_el1 → 00 MMU0
```

↓

```
00 PC 0000000000000000 TTBR0 000000
```

↓

```
bl early_map_kernel ← 0 init_pg_dir 0000000000
```

```
00 FDT000 KASLR 000000
```

```
map_kernel() 000000 KIMAGE_VADDR+offset:
```

```
_text → _stext : PAGE_KERNEL (non-exec)
```

```
_stext → _etext : PAGE_KERNEL_ROX (000)
```

```

| | |      __start_rodata   : PAGE_KERNEL (0000) |
| | |
| | |      __inittext       : PAGE_KERNEL_ROX (init00) |
| | |
| | |      _data → _end     : PAGE_KERNEL (0000) |
| | |
| | |      TTBR1 ← init_pg_dir → swapper_pg_dir00000000 |
| | |
| | |      ↓
| | |      |
| | |      ldr x8, =__primary_switched ← 00000000 |
| | |
| | |      br x8                                ← 0000000000000000 |
| | |

```

```

| | |
| | |      ▼
| | |
| | |      003: C00 → 0000
| | |

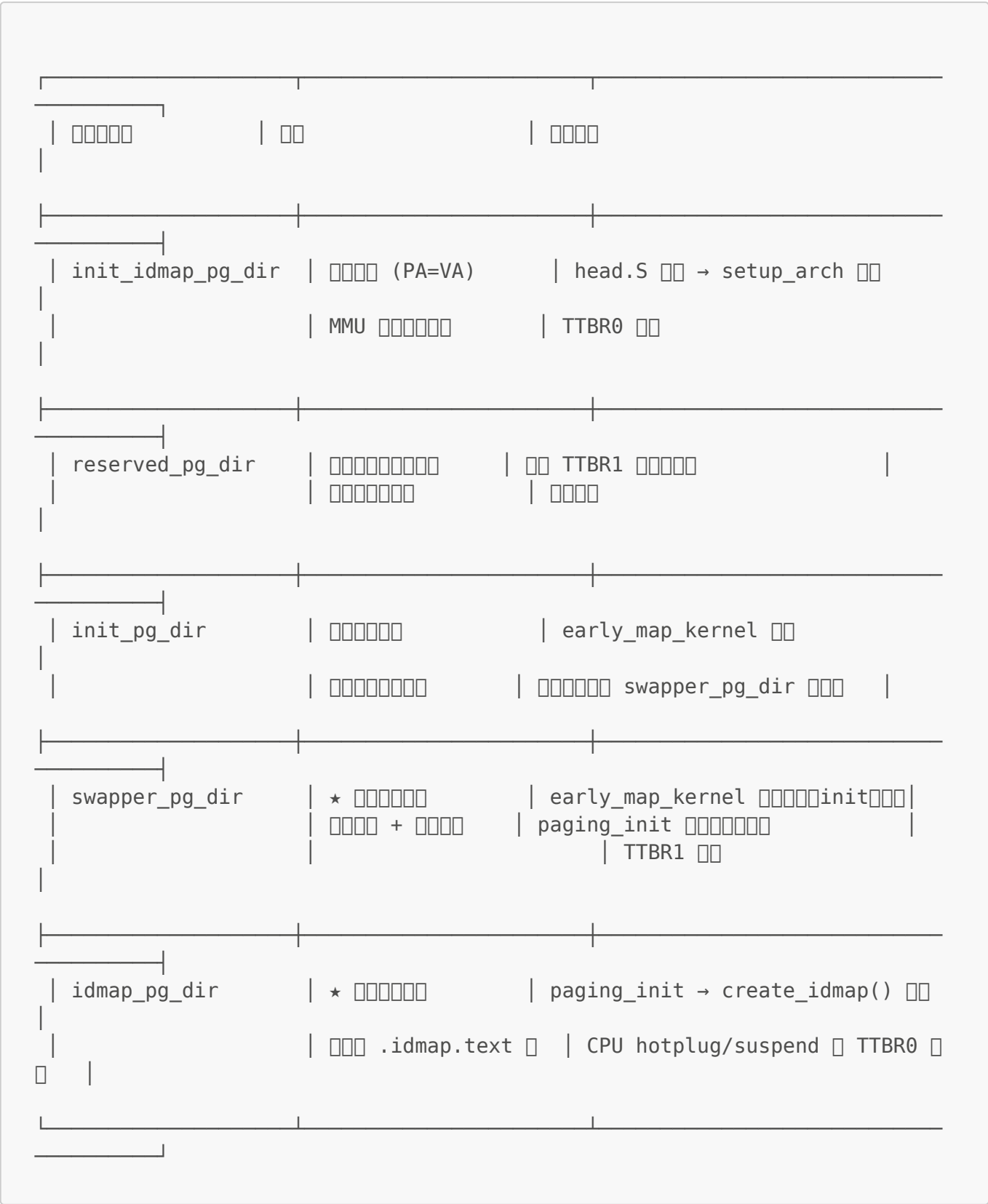
```

```

| | |
| | |      __primary_switched:
| | |      |
| | |      00 kimage_voffset = _text(VA) - _text(PA) |
| | |
| | |      bl start_kernel
| | |      |
| | |      → setup_arch()
| | |      |
| | |      → cpu_uninstall_idmap() ← TTBR0 000000000000 |
| | |      → arm64_memblock_init() ← 00000000 memblock |
| | |
| | |      → paging_init()          ← * 00000000 |
| | |
| | |      map_mem(swapper_pg_dir) ← 00000000 |
| | |
| | |      0000 memblock.memory 00 |
| | |
| | |      __map_memblock() 0000 PAGE_OFFSET 000 |
| | |
| | |      create_idmap()      ← 000000 idmap_pg_dir |
| | |
| | |      declare_kernel_vmas() ← 0000 VMA 00 |
| | |
| | |      → bootmem_init()        ← zone/node 0000buddy 00 |
| | |

```

0.4



0.5 primary_entryarch/arm64/kernel/head.S:85

```

;;  boot
;; x19 = MMU  EFI stub  MMU
;; x20 = CPU boot mode  EL1 or EL2
;; x21 = FDT

SYM_CODE_START(primary_entry)
    bl    record_mmu_state      ;;  CurrentEL  MMU  x19
    bl    preserve_boot_args    ;;  x0-x3  bootloader  boot_args[]

    ;;  C
    adrp  x1, early_init_stack
    mov   sp, x1
    mov   x29, xzr              ;; frame pointer = 0

    ;; *
    adrp  x0, __pi_init_idmap_pg_dir    ;; x0 =
    mov   x1, xzr                      ;; clrmask = 0
    bl    __pi_create_init_idmap        ;;  map_range()

    ;; Cache
    cbnz  x19, 0f                  ;;  MMU  clean
    dmb   sy                       ;; MMU  invalidate cache
    ...dcache_inval_poc...        ;;
    b     1f
0:  ...dcache_clean_poc...        ;; MMU  clean .idmap.text
1:  mov   x0, x19
    bl    init_kernel_el          ;;  EL1  EL2  boot mode
    mov   x20, x0

    bl    __cpu_setup              ;; *  MMU
    b     __primary_switch        ;;  MMU  +
SYM_CODE_END(primary_entry)

```

0.6 __cpu_setup arch/arm64/mm/proc.S:465

```

SYM_FUNC_START(__cpu_setup)
    tlbi  vmlle1                  ;;  TLB
    dsb   nsh

    ;; *  MAIR_EL1
    ;;  5
    ;;  MT_DEVICE_nGnRnE (idx=0) :
    ;;  MT_DEVICE_nGnRE  (idx=1) :  Early Write Ack
    ;;  MT_NORMAL_NC     (idx=2) :
    ;;  MT_NORMAL        (idx=3) : *  Write-Back
    ;;  MT_NORMAL_TAGGED (idx=4) : MTE
    mov_q  mair, MAIR_EL1_SET
    msr   mair_el1, mair

```

```

;; *  TCR_EL1
;; TCR_T0SZ = 64 - IDMAP_VA_BITS    → TTBR0
;; TCR_T1SZ = 64 - VA_BITS_MIN(48) → TTBR1
;; TCR_TG0  = 4KB granule           → TTBR0
;; TCR_TG1  = 4KB granule           → TTBR1
;; TCR_SHARED | TCR_CACHE_FLAGS    → Inner/Outer WB-WA cacheable
;; TCR_IPS   = MMFR0              →
mov_q tcr, TCR_T0SZ(...) | TCR_T1SZ(...) | ...
msr tcr_el1, tcr

;;  SCTLR_EL1__enable_mmu
mov_q x0, INIT_SCTLR_EL1_MMU_ON
ret
;;  head.Sx0 = SCTLR
SYM_FUNC_END(__cpu_setup)

```

0.7 __primary_switch + __enable_mmu

```

SYM_FUNC_START_LOCAL(__primary_switch)
    adrp x1, reserved_pg_dir        ;; x1 = TTBR1
    adrp x2, __pi_init_idmap_pg_dir  ;; x2 = TTBR0
    bl  __enable_mmu                ;; * MMU
    ;; — CPU —
    ;;  TTBR0 PA=VA
    adrp x1, early_init_stack
    mov  sp, x1
    mov  x29, xzr
    mov  x0, x20                    ;; boot status
    mov  x1, x21                    ;; FDT
    bl  __pi_early_map_kernel        ;; *
    ldr  x8, =__primary_switched     ;; *
    adrp x0, KERNEL_START            ;; __pa(KERNEL_START)
    br   x8                         ;; *
SYM_FUNC_END(__primary_switch)

;; -----

SYM_FUNC_START(__enable_mmu)
    ;;
    mrs  x3, ID_AA64MMFR0_EL1
    ubfx x3, x3, #TGRAN_SHIFT, 4
    cmp  x3, #TGRAN_SUPPORTED_MIN
    b.lt __no_granule_support        ;;
    ;; * TTBR
    phys_to_ttbr x2, x2
    msr  ttbr0_el1, x2                ;; TTBR0 ← init_idmap_pg_dir
    load_ttbr1 x1, x1, x3             ;; TTBR1 ← reserved_pg_dir

    ;; ***  SCTLR_EL1 MMU ***

```

```

    set_sctlr_el1 x0                ;; M=1, C=1, I=1
    ret                            ;; CPU 00000000
SYM_FUNC_END(__enable_mmu)

```

0.8 000000early_map_kernelarch/arm64/kernel/pi/map_kernel.c:241

```

asmlinkage void __init early_map_kernel(u64 boot_status, phys_addr_t fdt)
{
    /* 1. 00 BSS + 0000 */
    memset(__bss_start, 0, (char *)init_pg_end - (char *)__bss_start);

    /* 2. 00 FDT 00 CPU feature overrides */
    void *fdt_mapped = map_fdt(fdt); // 00000000 FDT
    init_feature_override(boot_status, fdt_mapped, chosen);

    /* 3. 0000 VA_BITS000000000000 */
    int va_bits = VA_BITS; // 52
    int root_level = 4 - CONFIG_PGTABLE_LEVELS; // 4 - 5 = -1

    /* 4. KASLR 00000000 */
    u64 kaslr_offset = pa_base % MIN_KIMG_ALIGN;
    if (CONFIG_RANDOMIZE_BASE) {
        kaslr_seed = kaslr_early_init(fdt_mapped, chosen);
        kaslr_offset |= kaslr_seed & ~(MIN_KIMG_ALIGN - 1);
    }

    /* 5. LPA2 00000052-bit PA 0000 TCR.DS 00 */
    if (CONFIG_ARM64_LPA2 && va_bits > VA_BITS_MIN)
        remap_idmap_for_lpa2();

    /* 6. * 000000000000 */
    u64 va_base = KIMAGE_VADDR + kaslr_offset;
    map_kernel(kaslr_offset, va_base - pa_base, root_level);
}

```

0.9 000000map_kernel — 00000000

```

static void __init map_kernel(u64 kaslr_offset, u64 va_offset, int
root_level)
{
    phys_addr_t pgdp = (phys_addr_t)init_pg_dir + PAGE_SIZE;

    /* 0000000000 init_pg_dir */
    map_segment(init_pg_dir, &pgdp, va_offset,
                _text, _stext, PAGE_KERNEL, ...); // 000000
    map_segment(init_pg_dir, &pgdp, va_offset,
                _stext, _etext, PAGE_KERNEL_ROX, ...); // * 0000000000
    map_segment(init_pg_dir, &pgdp, va_offset,
                __start_rodata, __inittext_begin,

```



```

        PAGE_KERNEL, ...); // 000000000000
map_segment(init_pg_dir, &pgdp, va_offset,
            __inittext_begin, __inittext_end,
            PAGE_KERNEL_ROX, ...); // init 00
map_segment(init_pg_dir, &pgdp, va_offset,
            __initdata_begin, __initdata_end,
            PAGE_KERNEL, ...); // init 00
map_segment(init_pg_dir, &pgdp, va_offset,
            _data, _end, PAGE_KERNEL, ...); // 000000

dsb(ishst);
idmap_cpu_replace_ttbr1((phys_addr_t)init_pg_dir); // * TTBR1 000

/* 0000000000 SCS 0000000000 */
if (twopass) {
    relocate_kernel(kaslr_offset);
    /* 0000000000000000 */
    unmap_segment(..., _stext, _etext, ...);
    map_segment(..., _stext, _etext, PAGE_KERNEL_ROX, ...);
}

/* * 00000000 swapper_pg_dir000 TTBR1 */
memcpy(swapper_pg_dir + va_offset, init_pg_dir, PAGE_SIZE);
dsb(ishst);
idmap_cpu_replace_ttbr1((phys_addr_t)swapper_pg_dir);
}

```

0.10 000000__primary_switched → start_kernel 000

```

SYM_FUNC_START_LOCAL(__primary_switched)
;; 0000000000000000TTBR1 = swapper_pg_dir
adr_l x4, init_task
init_cpu_task x4, x5, x6 ;; 00 init_task 00000

adr_l x8, vectors
msr vbar_el1, x8 ;; 0000000000000000

str_l x21, __fdt_pointer ;; 00 FDT 00000000

;; * 00000000-00000
adrp x4, _text ;; x4 = _text 0000
sub x4, x4, x0 ;; x0 = KERNEL_START 0000
str_l x4, kimage_voffset ;; kimage_voffset = VA - PA

bl finalise_el2 ;; 0000 EL2/VHE 00
bl start_kernel ;; *** 00 C 00000
ASM_BUG() ;; 0000
SYM_FUNC_END(__primary_switched)

```

0.11 000000paging_init — 0000

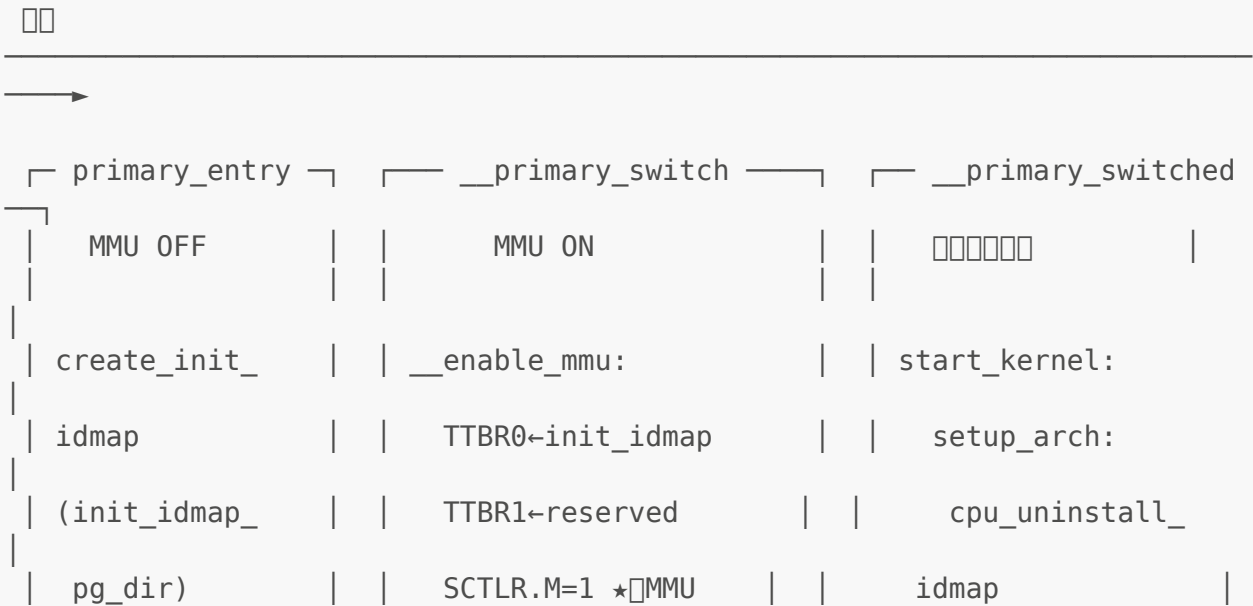
```
/* setup_arch() 初始化 (arch/arm64/kernel/setup.c:332-344) */
void __init setup_arch(char **cmdline_p)
{
    cpu_uninstall_idmap();      // ★ TTBR0 初始化
    arm64_memblock_init();     // 初始化 memblock
    paging_init();             // ★ 初始化
    bootmem_init();            // zone/node 初始化 → buddy 系统
}

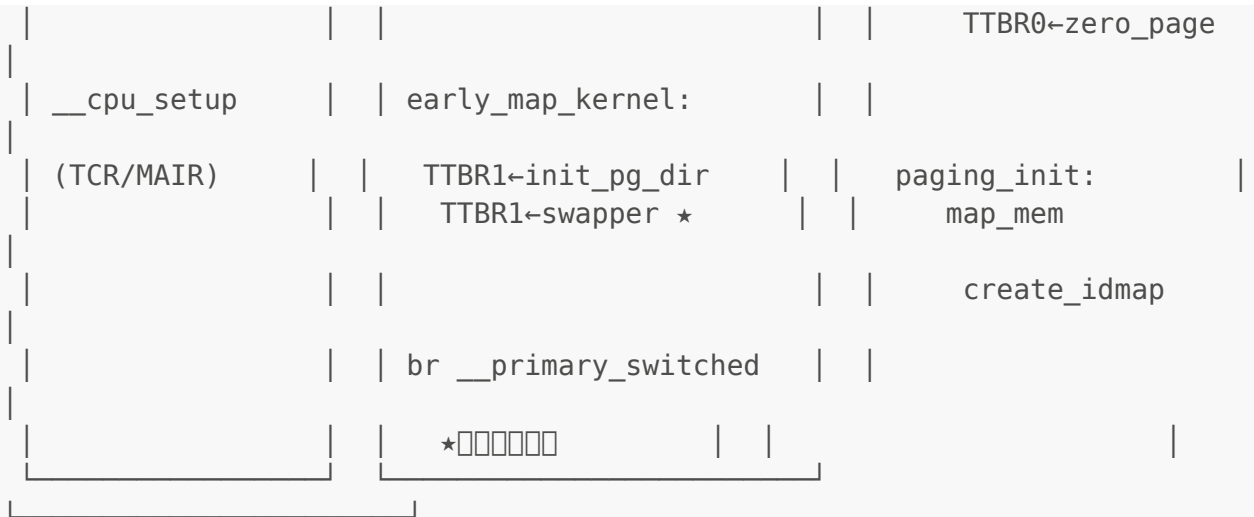
/* paging_init() (arch/arm64/mm/mmu.c:1351) */
void __init paging_init(void)
{
    /* 1. 初始化 */
    map_mem(swapper_pg_dir);
    // 初始化 memblock.memory 系统
    // __map_memblock(pgd, start, end, PAGE_KERNEL)
    // 设置 PAGE_OFFSET
    // 初始化 NOMAP 区域 non-exec

    /* 2. 初始化 idmap */
    memblock_allow_resize();
    create_idmap();
    // 设置 __idmap_text_start → __idmap_text_end
    // 初始化 idmap_pg_dir 系统 init_idmap_pg_dir
    // 初始化 CPU suspend/hotplug

    /* 3. 初始化 VMA */
    declare_kernel_vmas();
    // 初始化 vm_struct
    // _text→_etext, rodata, inittext, initdata, _data→_end
}
```

0.12 初始化 TTBR0 / TTBR1 系统





TTBR0: [init_idmap_pg_dir] → [zero_page] 00000000
 TTBR1: [reserved_pg_dir] → [init_pg_dir] → [swapper_pg_dir] 00000000

0.13 create_init_idmap 00000000pi/map_range.c

```

/* 0000000000000000 */
void map_range(phys_addr_t *pte, u64 start, u64 end, phys_addr_t pa,
               pgprot_t prot, int level, pte_t *tbl, ...)
{
    // level: 0=PGD, 1=PUD, 2=PMD, 3=PTE
    // lshift = (3 - level) * TABLE_SHIFT

    while (start < end) {
        u64 next = min(align_up(start, block_size), end);

        if (level < 2 || 00 block 00) {
            // 00000000 → 0000000000
            if (pte_none(*tbl)) {
                *tbl = 000 | PMD_TYPE_TABLE;
                *pte += PAGE_SIZE; // 0000000000
            }
            // 00000000
            map_range(pte, start, next, pa, prot, level+1, ...);
        } else {
            // 00000000 → 0000 block/page 00
            *tbl = pa | protval; // 2MB block 0 4KB page
        }
        start = next; tbl++;
    }
}

```

000000

- MMU 00000000000000000000 __pa()/__va() 00
- 00000000000000000000 ptep 0 pg_dir + PAGE_SIZE 0000

- `__pi_` `Position Independent` `MMU off`

0.14 QEMU 0.1 GDB MMU

```
#  __enable_mmu
(gdb) hbreak *__enable_mmu
(gdb) c

# 
(gdb) info reg x0 x1 x2
# x0 = SCTLR M=1 
# x1 = reserved_pg_dir → TTBR1
# x2 = init_idmap_pg_dir → TTBR0

#  TCR __cpu_setup 
(gdb) p/x $TCR_EL1
#  T0SZ, T1SZ, TG0, TG1, IPS 

# 
(gdb) x/8gx $x2
#  : PGD[0] | PMD_TYPE_TABLE

#  set_sctlr_el1 hbreak 
(gdb) si
#  *  MMU 

# 
(gdb) hbreak *__primary_switch+48 # br x8 
(gdb) c
(gdb) info reg x8
# x8 = 0xffff800080xxxxx__primary_switched 
(gdb) si
#  *  PC →
```

0.15 QEMU 0.2 GDB

```
#  paging_init TTBR1
(gdb) b paging_init
(gdb) c

#  TTBR1 swapper_pg_dir 
(gdb) p/x $TTBR1_EL1

#  map_mem 
(gdb) b __map_memblock
(gdb) c
(gdb) p/x start
(gdb) p/x end
#  0x40000000 → 0x80000000 = 1GB
```

```
# 查看 swapper_pg_dir 地址
(gdb) x/8gx &swapper_pg_dir
# map_mem 地址 PGD 地址
```

0.16 QEMU 0.3 设置 kimage_voffset

```
# kimage_voffset = 0 - 0
(gdb) b __primary_switched
(gdb) c

# 查看地址
(gdb) p/x $x4      # _text 地址
(gdb) p/x $x0      # KERNEL_START 地址
(gdb) p/x $x4 - $x0  # 计算 kimage_voffset

# 设置 kimage_voffset
(gdb) p/x kimage_voffset
# 设置 0xffff7fffc0000000 QEMU 地址

# 设置 start_kernel - kimage_voffset = 0
(gdb) p/x &start_kernel
(gdb) p/x &start_kernel - kimage_voffset
# 设置 QEMU 的 RAM 地址 0x40000000+
```

0.17 流程

- 设置 primary_entry → __cpu_setup → __primary_switch → __enable_mmu → early_map_kernel → __primary_switched → start_kernel 流程
- 设置 MMU 地址 PC 地址
- 设置 TTBR0/TTBR1 地址
- 设置
- 设置 __cpu_setup 地址 MAIR 地址 TCR 地址 SCTLR 地址
- 设置 ldr x8, =__primary_switched; br x8 地址
- 设置 paging_init() 地址 early 地址
- 设置 QEMU/GDB 地址 MMU 地址

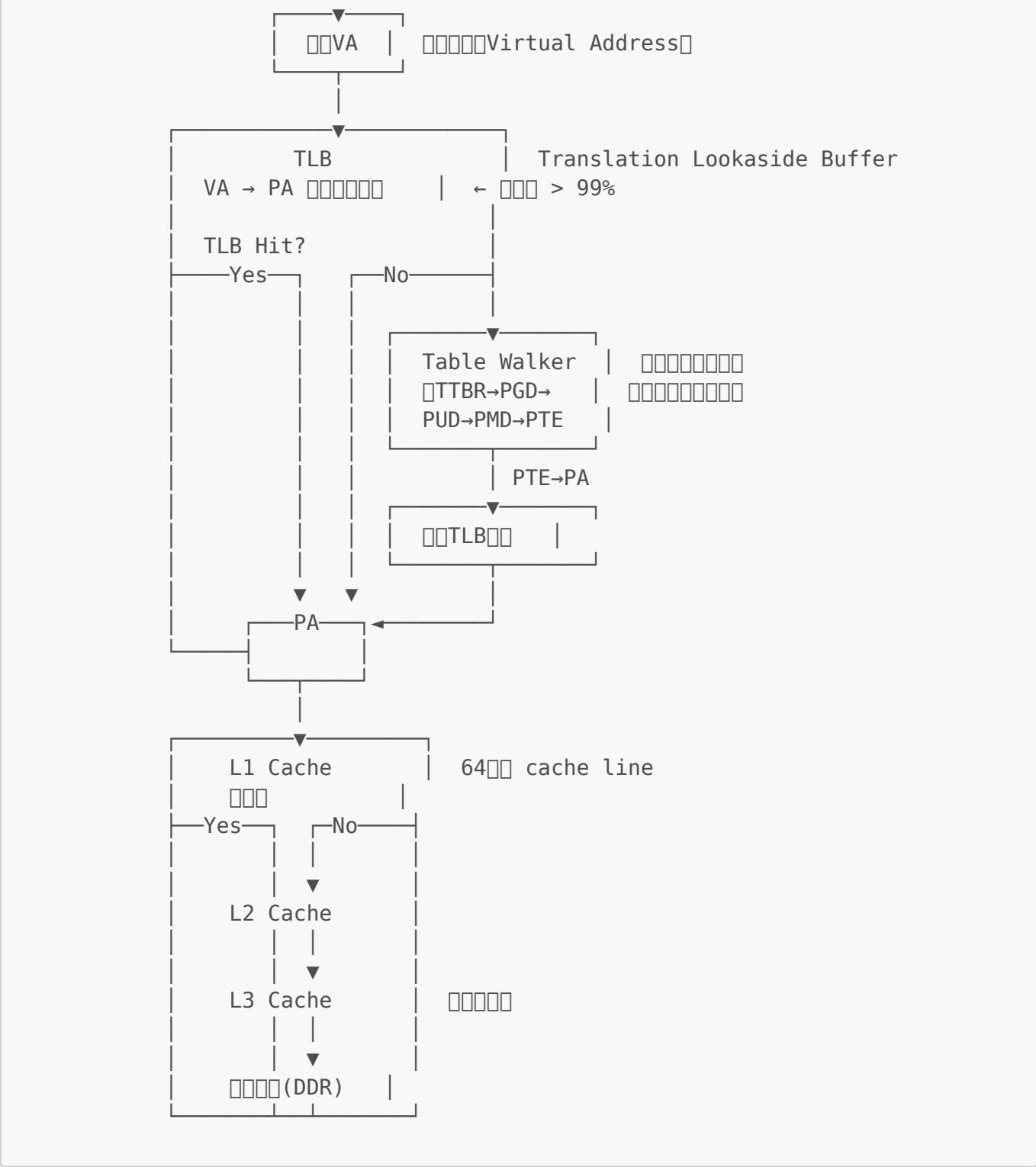
1.1 ARM64 设置 3.1

1.1 设置

ARM64 的 MMU 和 Cache 和 TLB 设置

1.2 设置 CPU 设置

CPU Core
|



1.3 ARM64 寄存器

寄存器	作用	相关函数/说明
TTBR0_EL1	EL0 虚拟地址到物理地址的转换表基址	switch_mm() 切换 MMU 时更新
TTBR1_EL1	EL1 虚拟地址到物理地址的转换表基址	通常指向 EL0 的 TTBR0_EL1
TCR_EL1	EL1 虚拟地址到物理地址的转换控制寄存器	__cpu_setup 配置
MAIR_EL1	EL1 内存属性索引寄存器，用于指定 Normal/Device/... 内存的属性	PTE 的 AttrIndx 字段
SCTLR_EL1	EL1 系统控制寄存器，控制 MMU 和 Cache	SCTLR_EL1.M=1 开启 MMU

CTLR

ELO

Cache line size

CTR_ELO

Cache line size

`cache_line_size()`

1.4 QEMU 1.1 MMU

```
# GDB
(gdb) break __primary_switch
(gdb) continue

# MMU
(gdb) info registers SCTLR_EL1 # MMU bit
# :
(gdb) p/x $SCTLR_EL1

# MMU
(gdb) stepi
```

1.5 QEMU 1.2 cache

```
# QEMU
cat /sys/devices/system/cpu/cpu0/cache/index0/size # L1 D-Cache
cat /sys/devices/system/cpu/cpu0/cache/index0/coherency_line_size # = 64
cat /sys/devices/system/cpu/cpu0/cache/index0/ways_of_associativity
cat /sys/devices/system/cpu/cpu0/cache/index0/number_of_sets
```

1.6

- TTBR/
- TLB miss Table Walker ARM64
- Cache line 64
- MAIR Normal-WB Device-nGnRnE MMIO
- QEMU cache

2.1

2.1

""

2.2

```
graph LR
  A[""]
```

```

DDR[0] --> B["Bootloader
(U-Boot)"]
    B --> | "DTB
memory[0] | C["DTB
[0]"]
    C --> | "early_init_dt_scan_memory()" | D["memblock
[0]"]
    D --> | "memblock_add()" | E["memblock.memory
[0]"]
    D --> | "memblock_reserve()" | F["memblock.reserved
[0]"]

style D fill:#4CAF50,color:#fff
style E fill:#8BC34A,color:#000
style F fill:#FF5722,color:#fff

```

2.3 内存块

```

struct memblock
├── struct memblock_type memory                ← 内存块
│   ├── cnt = 1                               ← 内存块QEMU 1GB 内存
│   ├── max = 128                             ← 内存
│   └── regions[]
│       └── [0]: base=0x40000000, size=0x40000000 (1GB)
└── struct memblock_type reserved              ← 保留内存
    ├── cnt = N
    └── regions[]
        ├── [0]: base=0x40080000, size=xxx    ← 内存
        ├── [1]: base=xxx, size=xxx           ← DTB
        └── [2]: base=xxx, size=xxx           ← initrd

内存块 = memory - reserved

```

2.4 内存块

```

1. arch/arm64/kernel/setup.c:setup_arch()
   └── arm64_memblock_init()                  ← 内存块
2. arch/arm64/mm/init.c
   ├── arm64_memblock_init()                  ← 内存 + 内存
   └── bootmem_init()                         ← sparse_init + zone_sizes_init
3. mm/memblock.c
   ├── memblock_add()                        ← 内存块
   ├── memblock_reserve()                   ← 内存块
   └── memblock_free_all()                   ← * 内存 buddy

```

2.5 QEMU 2.1 GDB 内存块


```
(gdb) break start_kernel
(gdb) continue
(gdb) p memblock.memory.cnt          # 000000
(gdb) p/x memblock.memory.regions[0] # 00000000
(gdb) p memblock.reserved.cnt        # 000000
(gdb) set $i = 0
(gdb) while $i < memblock.reserved.cnt
> p/x memblock.reserved.regions[$i]
> set $i = $i + 1
> end
```

2.6 QEMU 2.2 memblock 测试

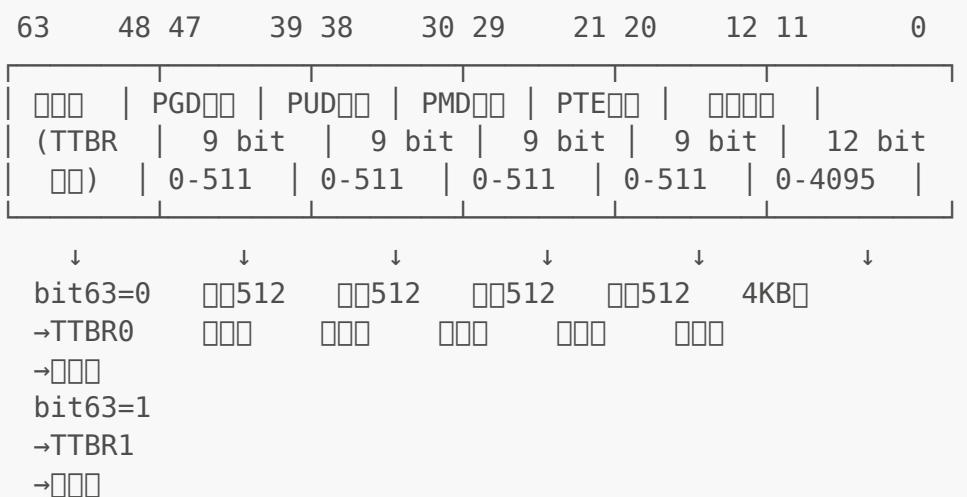
```
# 测试 memblock=debug
# QEMU 启动: -append "... memblock=debug"
# 查看 dmesg
dmesg | grep -i memblock | head -40
```

3 ARM64 5级页表

3.1 简介

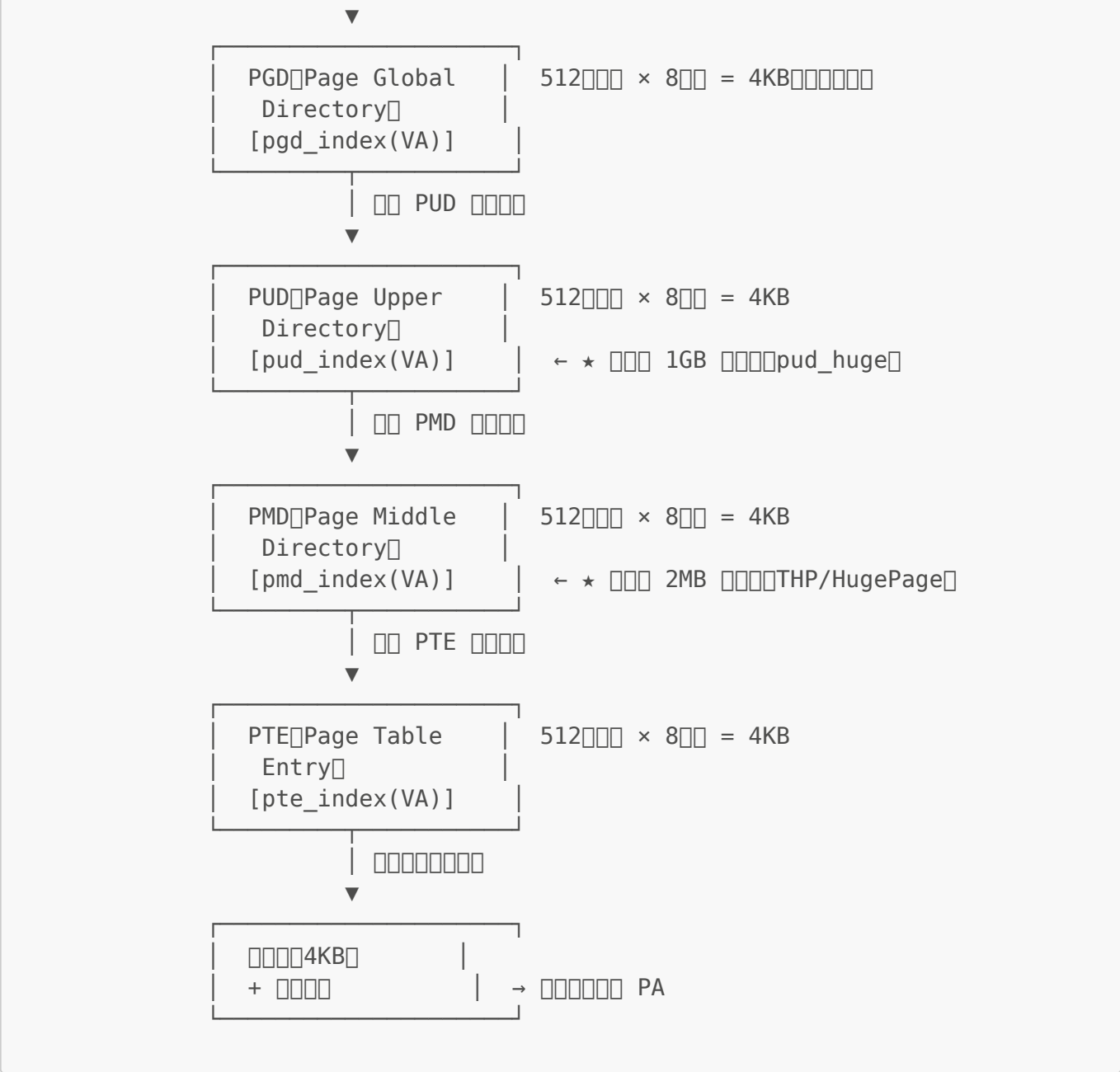
ARM64 5级页表 VA 到 PA

3.2 5级页表 VA_BITS=48 4KB

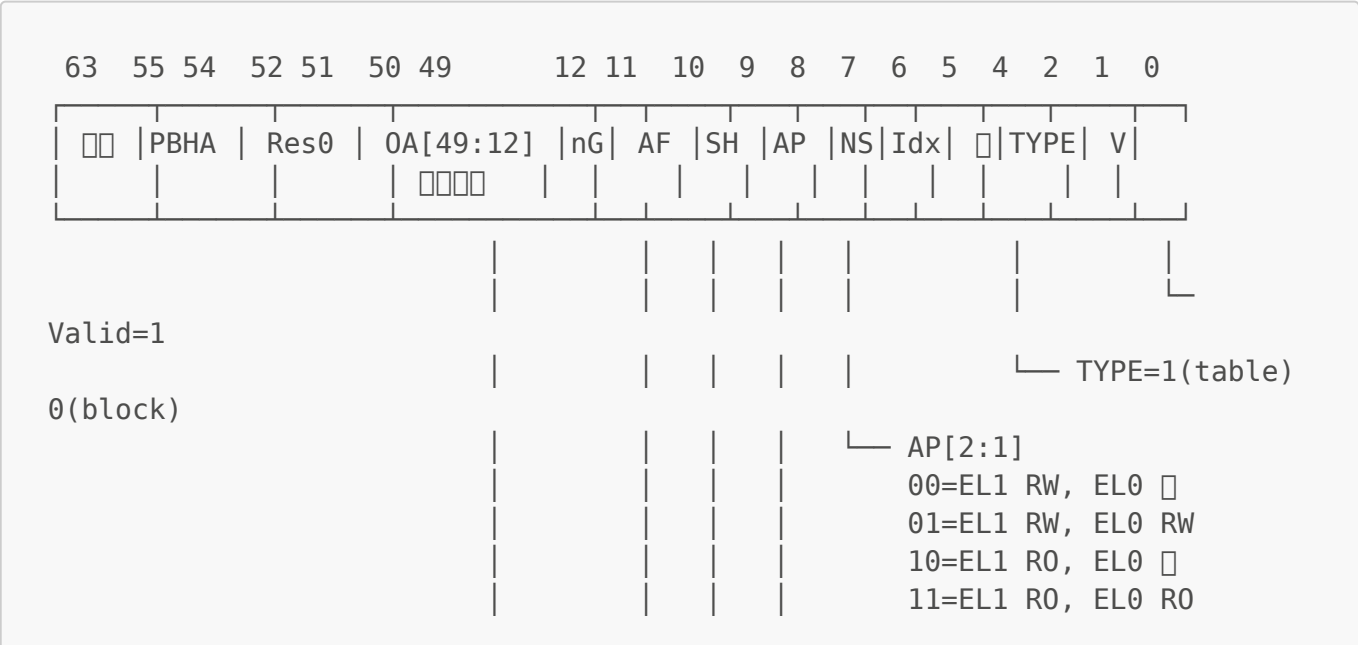


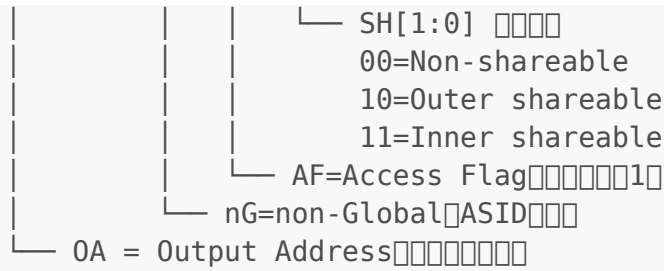
3.3 5级页表

TTBR0_EL1 PGD



3.4 PTE ARM64 Stage 1 4KB





3.5 内存管理

1. arch/arm64/include/asm/pgtable-hwdef.h ← PTE 定义
2. arch/arm64/include/asm/pgtable-types.h ← pgd_t/pud_t/pmd_t/pte_t 定义
3. arch/arm64/include/asm/pgtable.h ← pte_mkwrite/pte_wrprotect 定义
4. mm/memory.c: set_pte_at() ← PTE 设置
5. arch/arm64/include/asm/tlbflush.h ← TLB flush 定义

3.6 QEMU 3.1 GDB 内存管理

```

# 内存地址 addr = 0x400000000000 .text 开始
(gdb) break do_page_fault
(gdb) continue

# 获取 PGD 地址
(gdb) set $pgd_base = (unsigned long *)current->mm->pgd

# 计算索引
(gdb) set $addr = 0x4000000UL
(gdb) set $pgd_idx = ($addr >> 39) & 0x1ff
(gdb) set $pud_idx = ($addr >> 30) & 0x1ff
(gdb) set $pmd_idx = ($addr >> 21) & 0x1ff
(gdb) set $pte_idx = ($addr >> 12) & 0x1ff
(gdb) printf "PGD[%d] PUD[%d] PMD[%d] PTE[%d]\n", $pgd_idx, $pud_idx,
$pmd_idx, $pte_idx

# 打印 PGD
(gdb) set $pgd_entry = $pgd_base[$pgd_idx]
(gdb) printf "PGD entry = %lx\n", $pgd_entry

# 获取 PUD 地址
(gdb) set $pud_base = (unsigned long *)__va($pgd_entry &
0x0000FFFFFFFF000UL)
(gdb) set $pud_entry = $pud_base[$pud_idx]
(gdb) printf "PUD entry = %lx\n", $pud_entry

# ... 获取 PMD
(gdb) set $pmd_base = (unsigned long *)__va($pud_entry &
0x0000FFFFFFFF000UL)
(gdb) set $pmd_entry = $pmd_base[$pmd_idx]
(gdb) printf "PMD entry = %lx\n", $pmd_entry
(gdb) set $pte_base = (unsigned long *)__va($pmd_entry &

```

```
0x0000FFFFFFFFF000UL)
(gdb) set $pte_entry = $pte_base[$pte_idx]
(gdb) printf "PTE entry = %lx → PA = %lx\n", $pte_entry, ($pte_entry &
0x0000FFFFFFFFF000UL)
```

3.7 QEMU 3.2 AP

```
# → Permission Fault
# GDB
(gdb) break do_page_fault
(gdb) commands
> printf "ESR=%lx FAR=%lx\n", $x1, $x0
> end
# ESR EC = 0x24 (Data Abort, lower EL)
# ISS WnR
```

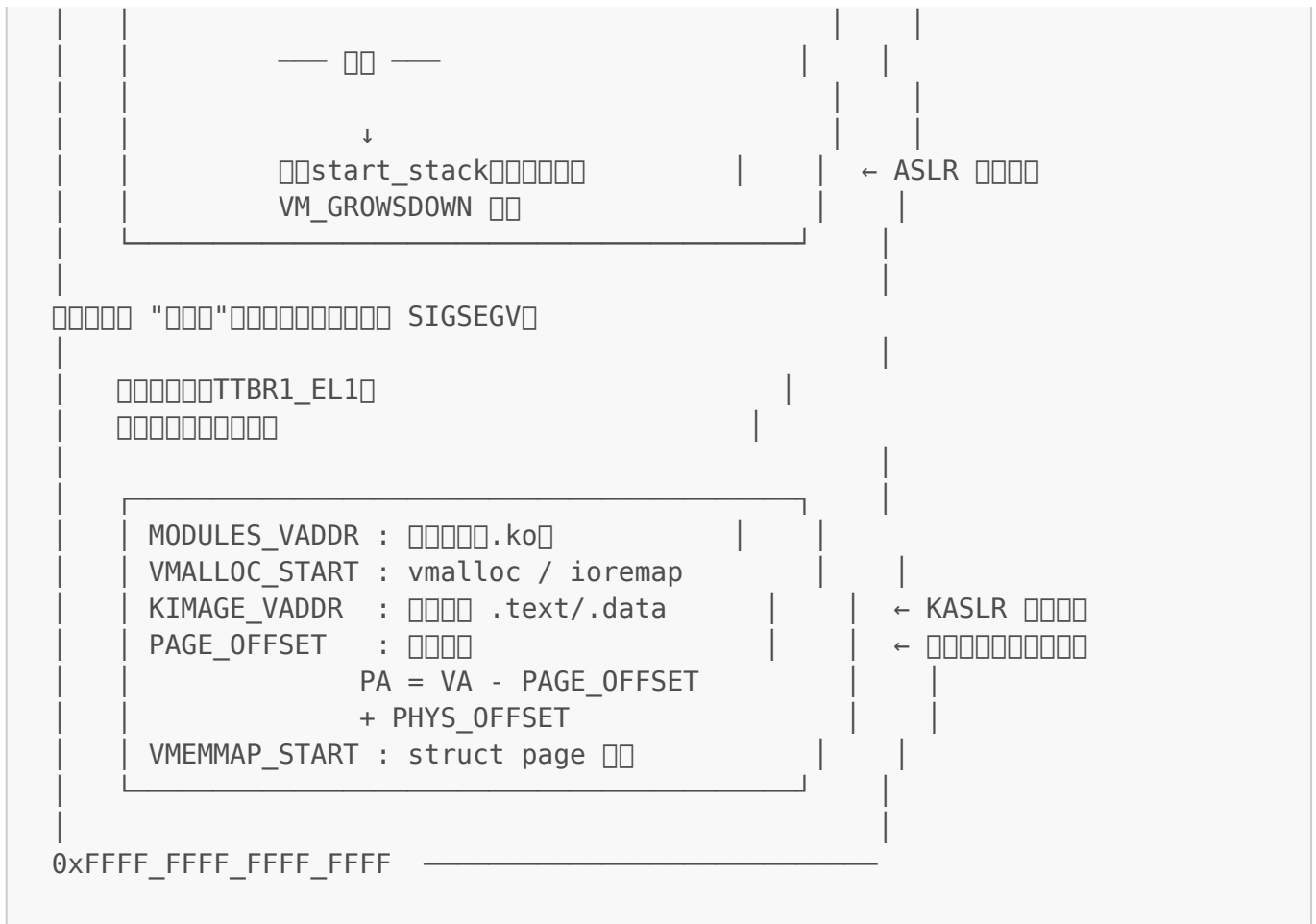
3.8

- VA → PGD_idx → PUD_idx → PMD_idx → PTE_idx → PA
- AP[2:1] VM_READ/VM_WRITE/VM_EXEC
- AF Access Flag LRU
- nG ASID TLB
- PMD block entry = 2MB THP

4 3

4.1 ARM64

```
0x0000_0000_0000_0000
├── TTBR0_EL1
│   └── TASK_SIZE = 1 << VA_BITS
│       └── 0x0000_0000_0000 : NULL
│           0x0000_0040_0000 : .text
│           0x0000_0060_0000 : .data + .bss
│               ↑
│               start_brk → brk
│               ─── ───
│               mmap_base ← mmap
│               ↓
│               /mmap
│               ← ASLR
```



4.2 QEMU 4.1 GDB 调试

```
(gdb) p/x PAGE_OFFSET
(gdb) p/x MODULES_VADDR
(gdb) p/x VMALLOC_START
(gdb) p/x KIMAGE_VADDR
(gdb) p/x current->mm->mmap_base      # mmap 开始
(gdb) p/x current->mm->start_brk      # 开始
(gdb) p/x current->mm->brk            # 结束
(gdb) p/x current->mm->start_stack     # 开始
```

4.3 QEMU 4.2 /proc/PID/maps 查看

```
# QEMU 进程的 VMA 信息
cat /proc/self/maps

# 开始
# 5580bad50000-5580bad52000 r-xp 00000000 ← .text[VM_READ|VM_EXEC]
# 5580baf51000-5580baf52000 rw-p 00001000 ← .data[VM_READ|VM_WRITE]
# 5580baf52000-5580baf73000 rw-p 00000000 ← [heap]
# 7f8a12340000-7f8a124d0000 r-xp 00000000 ← libc.so[mmap]
# 7ffcb3a21000-7ffcb3a42000 rw-p 00000000 ← [stack]
```

5 struct page / folio

5.1 struct page

struct page

64 C union

1 Buddy

```
. lru      → free_area[order].free_list
. private  → buddy order
. _mapcount → PAGE_BUDDY_MAPCOUNT_VALUE
```

2

```
. mapping  → anon_vma
. index    →
. _mapcount → PTE
. _refcount →
. lru      → LRU active/inactive
```

3 / Page Cache

```
. mapping  → address_space inode
. index    →
. private  → buffer_head /
```

4 SLUB Slab

```
. freelist  →
. counters  → inuse | objects | frozen
. slab_cache → kmem_cache
. slab_list  → node partial
```

5.2 QEMU 5.1 GDB page

```
# 1. buddy
(gdb) p sizeof(struct page)      # → 64

# 2.
(gdb) break do_anonymous_page
(gdb) continue
(gdb) finish
#
(gdb) p *vmf->page
(gdb) p page_count(vmf->page)    #
```

```
(gdb) p page_mapcount(vmf->page)      # PTE□□□

# 3. □□ slab □
(gdb) set $obj = kmalloc(256, 0xCC0)    # GFP_KERNEL
(gdb) set $page = virt_to_page($obj)
(gdb) p ((struct slab *)$page)->slab_cache->name  # □□□ "kmalloc-256"
(gdb) p ((struct slab *)$page)->inuse        # □□□□□
```

6 Buddy 系统 5

6.1 Buddy 系统

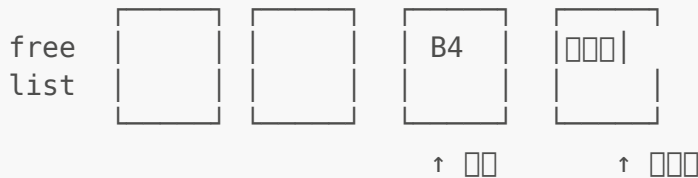
alloc_pages(order=2) 返回 4 个 2^2 页

order: 0(1页) 1(2页) 2(4页) 3(8页) 4(16页) ... 10(1024页)



order=2 页面 → 合并 order=3

A8 8 页 → A4 4 页 + B4 4 页 order=2 页面



合并 A4 4 页

free_pages(A4, order=2) 返回 4 页

合并 B4 4 页

→ 合并 A8 order=3 页面 order=3

→ 合并 A8 8 页 ... 合并

合并 buddy_pfn = pfn ^ (1 << order)

PFN=0x100, order=2 → buddy = 0x100 ^ 0x4 = 0x104

PFN 页 order 页

6.2 Migrate Type

free_area[order].free_list[MIGRATE_TYPE]

MIGRATE_UNMOVABLE ← 不可移动的内存

```

MIGRATE_MOVABLE      ← 可移动内存compaction
MIGRATE_RECLAIMABLE ← page cache可回收内存

fallback 内存
MOVABLE → RECLAIMABLE → UNMOVABLE不可移动内存

```

6.3 内存分配

```

1. mm/page_alloc.c: __alloc_pages()      ← 分配
2. mm/page_alloc.c: get_page_from_freelist() ← 从 zone 中 free_area 中
3. mm/page_alloc.c: __free_one_page()     ← 释放内存
4. mm/page_alloc.c: __rmqueue()           ← 从队列中
5. include/linux/mmzone.h: struct free_area ← 内存块

```

6.4 QEMU 中 6.1 中的 buddyinfo

```

# QEMU 中
cat /proc/buddyinfo

# 内存块大小 = 2^order 内存块大小
# Node 0, zone Normal    1234  567  234  89  42  18  7  3  1  0  0
#                          ↑    ↑    ↑    ↑    ↑
#                          order0 ord1 ord2   ord4      ord10

# 内存块大小
# 1234×1 + 567×2 + 234×4 + 89×8 + 42×16 + 18×32 + 7×64 + 3×128 + 1×256

```

6.5 QEMU 中 6.2 中的 GDB 内存分配

```

(gdb) break __rmqueue_smallest
(gdb) commands
> printf "zone=%s order=%d mi=%d\n", zone->name, $x1, $x2
> end

(gdb) break __free_one_page
(gdb) commands
> printf "free pfn=%lx order=%d → buddy_pfn=%lx\n", pfn, order,
pfn^(1<<order)
> end

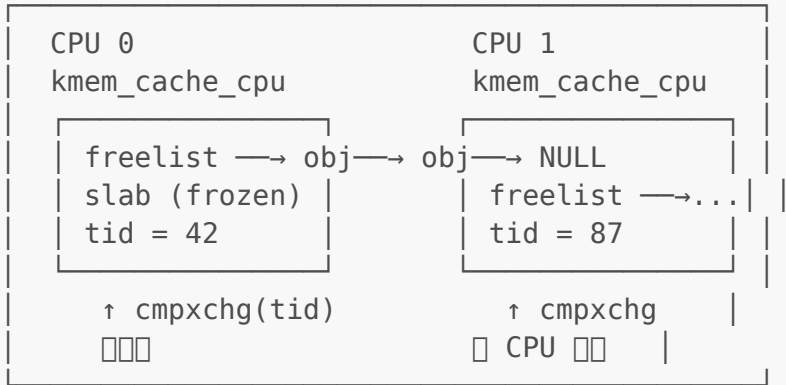
```

7 中的 SLUB 内存分配

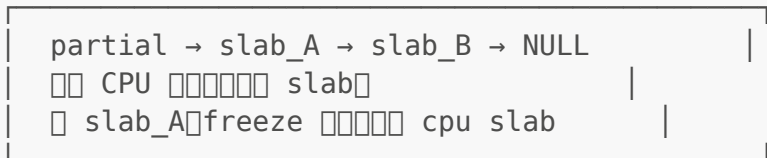
7.1 中的 SLUB 内存分配

kmem_cache_alloc() 函数

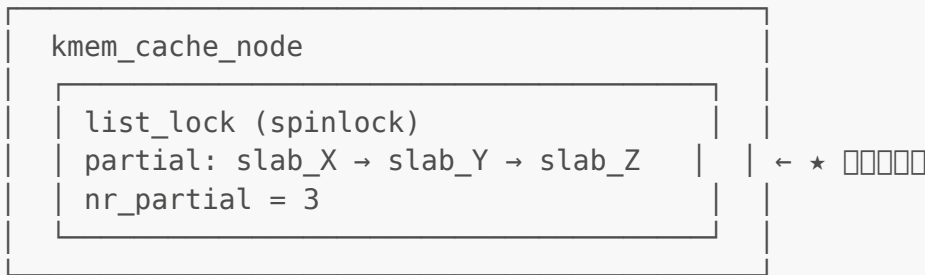
Step 1: 初始化 per-CPU freelist



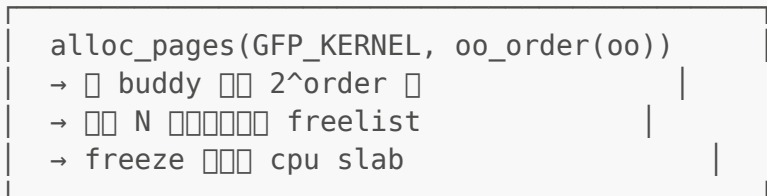
Step 2: 初始化 per-CPU partial list



Step 3: 初始化 node partial list



Step 4: Buddy 系统 slab



7.2 QEMU 7.1 版本

```
# GDB 命令
(gdb) break __slab_alloc_node
(gdb) commands
> printf "cache=%s\n", s->name
> printf " freelist=%p slab=%p\n", c->freelist, c->slab
```

```
> end
```

```
# 0000000000000000 open 00000000
# 0000000000000000
```

8 mmap VMA 5

8.1 mmap VMA

```
graph TD
    subgraph "mm_struct00000000"
        MT["maple_tree mm_mt"]
        A["VMA: .text  
0x400000-0x402000  
r-xp VM_READ|VM_EXEC"]
        B["VMA: .data  
0x600000-0x601000  
rw-p VM_READ|VM_WRITE"]
        C["VMA: heap  
0x601000-0x621000  
rw-p VM_READ|VM_WRITE"]
        D["VMA: libc.so  
0x7f...000-0x7f...130  
r-xp 0000"]
        E["VMA: stack  
0x7ffc...000-0x7ffc...000  
rw-p VM_GROWSDOWN"]
    end
    MT --> A
    MT --> B
    MT --> C
    MT --> D
    MT --> E

    A -->|"vm_file ≠ NULL"| F["inode: /usr/bin/xxx"]
    D -->|"vm_file ≠ NULL"| G["inode: /lib/libc.so"]
    C -->|"vm_file = NULL"| H["0000"]
    E -->|"vm_file = NULL"| I["0000"]
```

8.2 VMA

```
0000000000000000 0000000000000000 00
-----
malloc(4096)
└─ brk() 0000 ──> do_brk_flags() ──> 00 heap VMA
                    └─ vma_merge() 0
                       vm_area_alloc()
```

8.3 QEMU 8.1 strace mmap

9.1 □□□□□□□□□□

27 / 37

```

NUMA []]

K -->|"vm_file=NULL"| O["do_anonymous_page()"]
K -->|"vm_file≠NULL"| P["do_fault()"]

O --> Q{" " or " "}
Q -->|" "| R[" zero_page
[]]
Q -->|" "| S["alloc_pages()
[]]

P --> T{" or " "}
T -->|" "| U["do_cow_fault()
[]+[]"]
T -->|" "| V["do_shared_fault()
[] page cache"]
T -->|" "| W["do_read_fault()
[] page cache"]

L --> X["alloc_pages() + copy
[]"]

S --> Y["set_pte_at()
[] PTE"]
X --> Y
R --> Y
U --> Y
V --> Y
W --> Y

Y --> Z["
[]"]

style G fill:#f44336,color:#fff
style H fill:#f44336,color:#fff
style R fill:#4CAF50,color:#fff
style S fill:#2196F3,color:#fff
style Y fill:#FF9800,color:#fff

```

9.2 QEMU 9.1

```

# GDB
(gdb) break do_page_fault
(gdb) commands
> printf "=== PAGE FAULT ===\n"
> printf "FAR (fault addr) = %lx\n", $x0
> printf "ESR = %lx\n", $x1
> end

(gdb) break do_anonymous_page
(gdb) commands

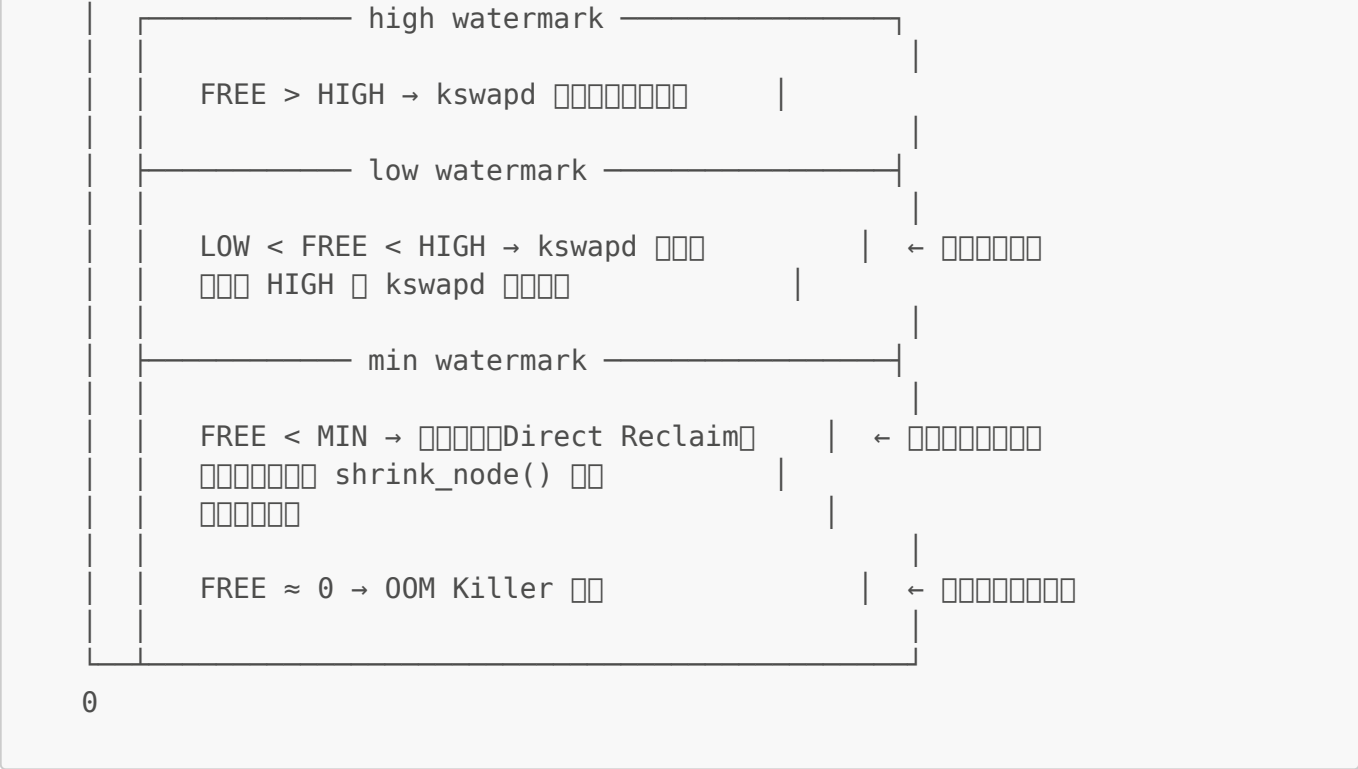
```

9.3 QEMU 9.2 COW

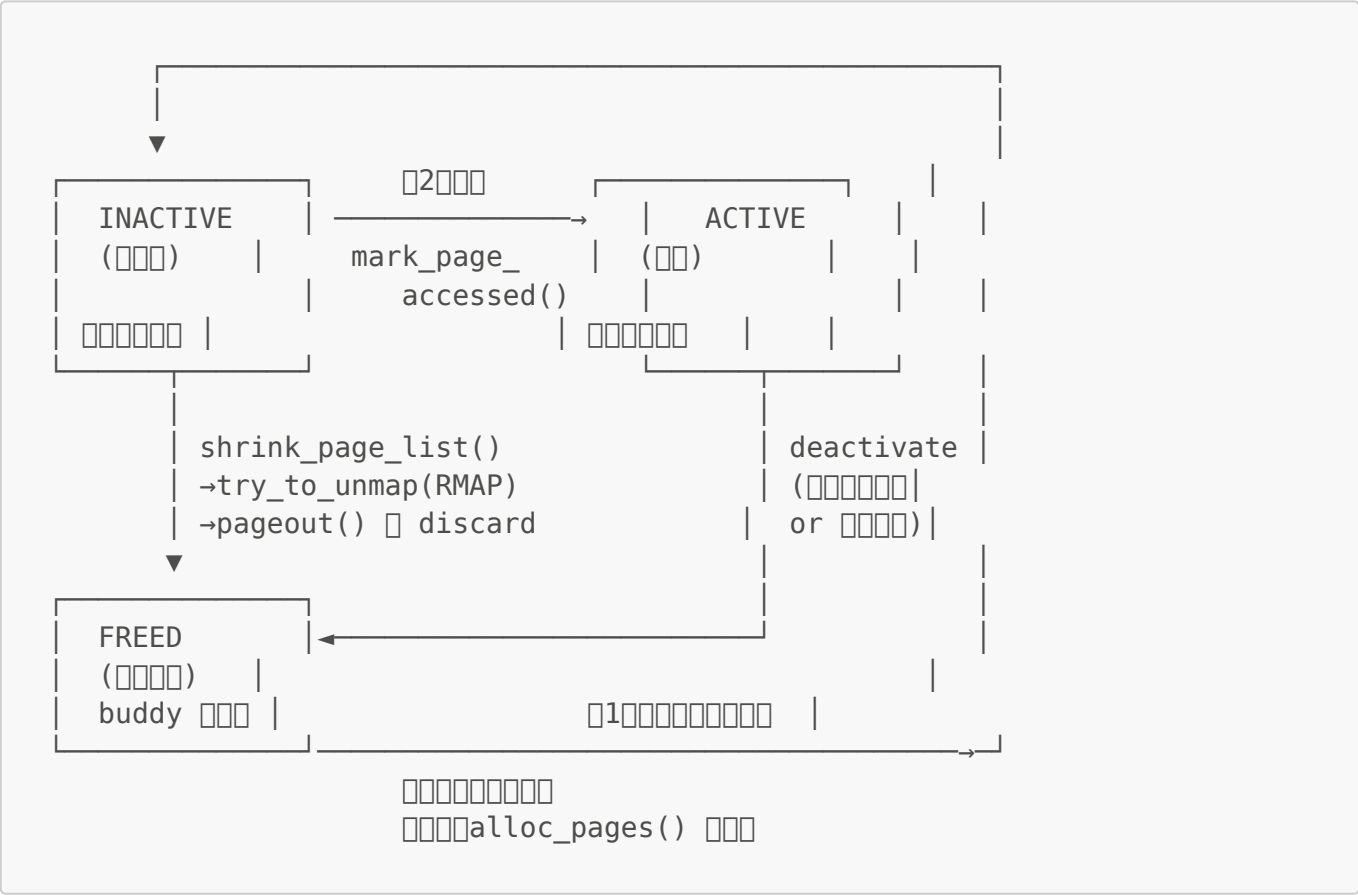
```
# GDB
(gdb) break do_wp_page
(gdb) break wp_page_copy
# COW
```

10.1

29 / 37

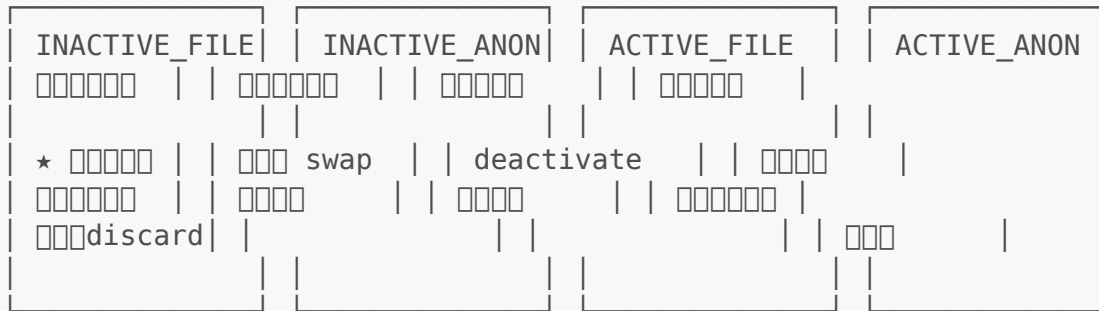


10.2 LRU



10.3





swappiness □□□0-200□□□60□□

0 → □□□ swap □□□□□□□□□□

60 → □□

200 → □□ swap □□□

10.4 □□□□□□

1. mm/vmscan.c: kswapd()	← □□□□□□□□□□
2. mm/vmscan.c: balance_pgdat()	← □□□□□□□□
3. mm/vmscan.c: shrink_node()	← □□□□ NUMA □□
4. mm/vmscan.c: shrink_lruvec()	← □□ LRU □□
5. mm/vmscan.c: shrink_page_list()	← □□□□□□
6. mm/rmap.c: try_to_unmap()	← RMAP □□□□□□
7. mm/swap.c: lru_cache_add()	← □□□□ LRU
8. mm/workingset.c	← □□□□□□

10.5 QEMU □□ 10.1□□□□□□ kswapd □□

```
# QEMU □□

# 1. □□ vmscan □□ ftrace □□
cd /sys/kernel/debug/tracing
echo 1 > events/vmscan/mm_vmscan_kswapd_wake/enable
echo 1 > events/vmscan/mm_vmscan_kswapd_sleep/enable
echo 1 > events/vmscan/mm_vmscan_lru_shrink_inactive/enable
echo 1 > events/vmscan/mm_vmscan_direct_reclaim_begin/enable
echo 1 > events/vmscan/mm_vmscan_direct_reclaim_end/enable
echo 1 > tracing_on

# 2. □□□□□□
# □□□C□□□□□□□□□□□□
cat > /tmp/pressure.c << 'EOF'
#include <stdlib.h>
#include <string.h>
int main() {
    for (int i = 0; i < 200; i++) {
        void *p = malloc(4 * 1024 * 1024); // □□4MB
        memset(p, i, 4 * 1024 * 1024);    // □□□□□□□□□□
    }
}
```

```

    return 0;
}
EOF
gcc -o /tmp/pressure /tmp/pressure.c
/tmp/pressure &

# 3. 查看
cat trace | tail -30

# 查看
# kswapd0-XX [000] mm_vmscan_kswapd_wake: nid=0 zid=2 order=0
# kswapd0-XX [000] mm_vmscan_lru_shrink_inactive: ... nr_scanned=32
nr_reclaimed=28

```

10.6 QEMU 和 10.2 GDB 断点设置

```

(gdb) break kswapd
(gdb) break shrink_lruvec
(gdb) commands
> p lruvec->lists[0]      # INACTIVE_ANON 链表
> p lruvec->lists[1]      # ACTIVE_ANON 链表
> p lruvec->lists[2]      # INACTIVE_FILE 链表
> p lruvec->lists[3]      # ACTIVE_FILE 链表
> end

(gdb) break shrink_page_list
(gdb) commands
> printf "nr_to_reclaim = %lu\n", sc->nr_to_reclaim
> end

```

11 内存管理进阶

11.1 THP 内存管理

普通 4KB 页 vs THP 2MB 大页 PMD block entry 对比

PGD → PUD → PMD → PTE → 4KB 页	PGD → PUD → PMD(block) → 2MB 大页
↑	↑
512 个 PTE	1 个 PMD entry
4KB 页	2MB 大页
512 次 TLB miss	1 次 TLB miss ← * 节省

普通 TLB 命中 × 512 次 TLB miss
 2MB 大页 compaction 节省

11.2 CMA 内存管理

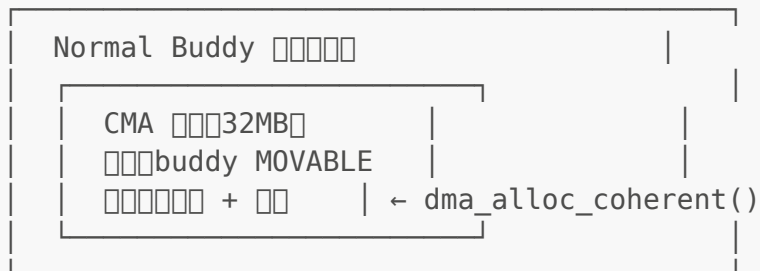
CMA Buddy

Buddy

CMA Contiguous Memory Allocator

MOVABLE buddy

DMA buffer → migrate_pages



11.3 OOM Killer

alloc_pages()

→ __alloc_pages_slowpath()

→ __alloc_pages_direct_reclaim() ←

→ __alloc_pages_direct_compact() ←

→ ?

→ __alloc_pages_may_oom()

→ out_of_memory()

→ select_bad_process() ← OOM score

→ oom_badness()

score = / * 1000

+ /proc/PID/oom_score_adj -1000 ~ 1000

→ oom_kill_process() ← SIGKILL

11.4 QEMU 11.1 OOM

QEMU 1GB

#

cat > /tmp/oom.c << 'EOF'

#include <stdio.h>

#include <stdlib.h>

#include <string.h>

int main() {

int i = 0;

while (1) {

char *p = malloc(10 * 1024 * 1024); // 10MB

```

        if (!p) break;
        memset(p, 0, 10 * 1024 * 1024);
        printf("Allocated %d MB\n", ++i * 10);
    }
    return 0;
}
EOF
gcc -o /tmp/oom /tmp/oom.c
/tmp/oom

# dmesg
dmesg | grep -i "oom\|killed\|out of memory"

```

12 内存管理

12.1 内存管理

```

// mm_lab.c - 内存管理
#include <linux/module.h>
#include <linux/mm.h>
#include <linux/slab.h>
#include <linux/mmzone.h>
#include <linux/swap.h>

static int __init mm_lab_init(void)
{
    struct page *page;
    struct zone *z;
    void *obj;

    pr_info("=== MM Lab Start ===\n");

    // 1: buddy
    page = alloc_pages(GFP_KERNEL, 2); // 4
    pr_info("[buddy] PFN=%lx zone=%s order=2\n",
            page_to_pfn(page), page_zone(page)->name);
    __free_pages(page, 2);

    // 2: SLUB
    obj = kmalloc(256, GFP_KERNEL);
    pr_info("[slub] obj=%px slab=%s\n",
            obj, virt_to_slab(obj)->slab_cache->name);
    kfree(obj);

    // 3: 水印
    z = &NODE_DATA(0)->node_zones[ZONE_NORMAL];
    pr_info("[watermark] min=%lu low=%lu high=%lu free=%lu\n",
            z->_watermark[WMARK_MIN],
            z->_watermark[WMARK_LOW],
            z->_watermark[WMARK_HIGH],

```

```

        zone_page_state(z, NR_FREE_PAGES));

// 004: LRU 00
pr_info("[lru] active_anon=%lu inactive_anon=%lu\n",
        node_page_state(NODE_DATA(0), NR_ACTIVE_ANON),
        node_page_state(NODE_DATA(0), NR_INACTIVE_ANON));
pr_info("[lru] active_file=%lu inactive_file=%lu\n",
        node_page_state(NODE_DATA(0), NR_ACTIVE_FILE),
        node_page_state(NODE_DATA(0), NR_INACTIVE_FILE));

pr_info("=== MM Lab End ===\n");
return -EINVAL; // 0000
}
module_init(mm_lab_init);
MODULE_LICENSE("GPL");

```

12.2 Makefile

```

obj-m := mm_lab.o
KDIR := /lib/modules/$(shell uname -r)/build

all:
    make -C $(KDIR) M=$(PWD) modules

clean:
    make -C $(KDIR) M=$(PWD) clean

```

12.3 0000000

```

#!/bin/bash
# mm_monitor.sh - 00000000
while true; do
    clear
    echo "===== $(date) ====="

    echo -e "\n--- Memory Overview ---"
    grep -E
    "MemTotal|MemFree|MemAvailable|Cached|Slab|PageTables|AnonPages"
    /proc/meminfo

    echo -e "\n--- Buddy Info ---"
    cat /proc/buddyinfo

    echo -e "\n--- Top SLUB caches ---"
    head -1 /proc/slabinfo
    sort -k3 -rn /proc/slabinfo | head -5

    echo -e "\n--- VM Stats (key counters) ---"
    grep -E

```

```
"pgfault|pgmajfault|pgsteal|pgscan|kswapd|pgalloc|pgfree|oom_kill"
/proc/vmstat

echo -e "\n--- Watermarks ---"
grep -A5 "Normal" /proc/zoneinfo | grep -E "pages free|min|low|high"

sleep 2
done
```

内存管理

序号	范围	描述	数量
1	1-2	内存块 + memblock	4
2	3	内存块 + 内存块	3
3	4-5	内存块 + struct page	4
4	6-7	Buddy + SLUB内存块	4
5	8-9	VMA + 内存块	5
6-7	10	内存块 kswapd/LRU内存块	4
8	11-12	内存块 + 内存块	5

内存管理

```
1. 2个内存块 + 内存块
2. 2个QEMU + GDB 内存块
3. 30个GDB 内存块 + 内存块
```

内存管理

1. 内存块
→ 在 GDB 中 sizeof()内存块
2. 内存块
→ 内存块happy path内存块
3. 内存块
→ "内存块" 在 "内存块" 中 10 个
4. 在 /proc 和 /sys 中
→ 内存块

Linux 6.18.1 ARM64 QEMU 1GB